类与对象的使用

讲师：余依函

目录

对象的初始化需求
Initialization Requirements

构造函数详解
Constructor Details

析构函数详解
Destructor Details

构造与析构的调用顺序
Invocation Order Analysis

虚拟现实中的应用案例
VR Application Cases

总结与回顾
Summary and Review

对象的初始化需求

THE NEED FOR OBJECT INITIALIZATION

为什么需要初始化？

核心问题：抽象与内存

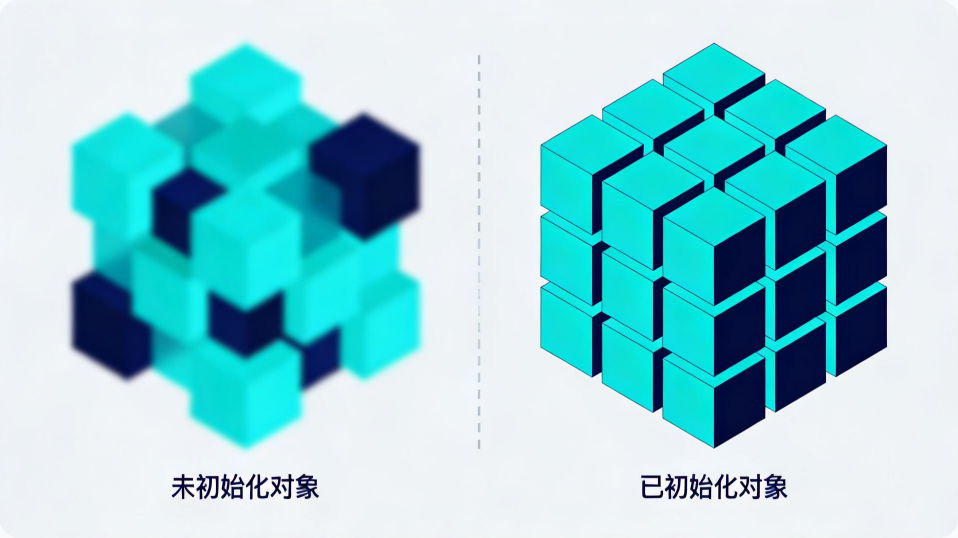
类是抽象的模板，本身不占用内存空间，因此无法在声明类时直接对数据成员进行赋值。

潜在风险：垃圾值与崩溃

未初始化的对象包含随机内存垃圾值，会导致程序行为不可预测，甚至引发严重的运行时错误。

解决方案：构造函数

C++ 提供构造函数机制，确保对象在创建瞬间被正确初始化，拥有确定的初始状态。



图示：未初始化对象（混乱） vs 已初始化对象（有序）

构造函数详解

DETAILED EXPLANATION OF CONSTRUCTORS

OBJECT INITIALIZATION & INSTANTIATION

构造函数的定义与特性

定义：构造函数是一种特殊的成员函数，它的唯一目的就是在创建对象时对其进行初始化。

命名规则

函数名必须与类名完全相同，大小写也需一致。

无返回值

不需要指定返回值类型，连 void 都不需要写。

自动调用

对象创建时自动执行，程序员不能手动调用。

默认构造

若未定义任何构造函数，编译器会生成一个空体的默认构造函数。

代码示例（C++）

```
classTime {// 定义Time类
public:
Time() {// 构造函数，无返回值，与类同名
hour = 0; minute = 0; sec = 0;// 初始化操作
cout <<"Time 对象已初始化。"<< endl;
}
voidshow_time();
inthour, minute, sec;
};
```

关键点：构造函数是对象生命周期的起点，负责赋予对象“生命”初始状态。

案例1：默认构造函数（例9.1）

这是教材中的经典案例，展示了如何使用默认构造函数初始化一个时间对象。

```
#include<iostream>
usingnamespacestd;

class Time {
public:
Time() {// 默认构造函数
hour = 0; minute = 0; sec = 0;
}
void set_time();
void show_time();
private:
int hour, minute, sec;
};

voidTime::show_time() {
cout << hour << ":" << minute << ":" << sec << endl;
}

Int main() {
Time t1;// 自动调用默认构造函数初始化
t1.show_time();// 显示初始时间 0:0:0
return0;
}
```

核心知识点：默认构造函数无参数，用于在对象创建时提供默认初始状态。

构造函数核心知识点总结

核心定义：构造函数是一种特殊的成员函数，主要用于在创建对象时初始化对象。

核心作用

在创建对象时，负责对数据成员进行初始化工作。

命名规则

构造函数的名字必须与它所属的类名完全一致。

主要特点

- 无返回类型
- 支持重载机制
- 可自定义或默认生成

调用时机

当对象进入其作用域时，由系统自动调用，无需手动触发。

💡 **关键点记忆：**构造函数是对象的“接生婆”，名字与类同，无返无值，自动调用，负责初始化。

带参数的构造函数

核心作用

突破默认构造函数的限制，允许在创建对象时传入特定数据，实现对象的**个性化初始化**。

语法格式

定义：类名(类型1 参数1, 类型2 参数2...) { 函数体 }
调用：类名 对象名(实参1, 实参2...);

代码示例（C++）

```
class Time {// 定义类
public:
Time(int h,int m,int s) {// 带参构造
hour = h; minute = m; sec = s;
}
void show_time();
};
Time t2(10,30,0);// 初始化对象
```

关键点总结：

1. 带参数的构造函数解决了默认构造函数无法初始化特定数据的问题。
2. 实参必须与形参的数量、类型和顺序严格匹配。

案例2：带参数的构造函数（例9.2）

核心概念解析

本案例通过定义Box类展示了带参数构造函数的使用。

1. **初始化**：在创建对象时，直接通过构造函数参数初始化长、宽、高，无需额外的setter方法。
2. **封装性**：将属性设为私有(private)，通过成员函数计算体积，保证了数据的安全性。

Box.cpp - C++ Code

```cpp
#include <iostream> usingnamespacestd;
class Box{
public:
Box(int h,int w,int len) :height(h), width(w), length(len) {}
int volume();
private:
int height, width, length;
};
int Box::volume() {return height * width * length; }
int main() {
Box box1(12,20,25);// 直接初始化尺寸
cout <<"Volume: "<< box1.volume() << endl;
return0;
}
```

带参数构造函数让对象的创建和初始化一步到位，减少了代码冗余，提升了程序的安全性与可读性。

举一反三案例1：VR场景中的物体初始化

应用场景

在VR游戏或应用开发中，加载3D模型并设置初始位置和大小是高频操作，直接影响场景构建效率。

解决方案

定义VRModel类，利用**带参数构造函数**封装模型路径、位置和缩放参数。创建对象时自动完成加载与初始化，实现数据与行为的统一。

```
class VRModel {
public:
// 带参数构造函数初始化模型
VRModel(const std::string& path,constglm::vec3& pos,floatscale)
: m_path(path), m_pos(pos), m_scale(scale) {
LoadModel(path);// 自动加载
}
private:
std::string m_path;
glm::vec3 m_pos;// 初始位置
floatm_scale;// 缩放比例
};

// 使用示例：创建即初始化
VRModel player("player.obj", glm::vec3(0,0,0), 1.0f);
VRModel enemy("enemy.obj", glm::vec3(5,0,0), 1.2f);
```

核心思想：利用构造函数封装对象的初始化逻辑，将“数据”与“初始化行为”绑定，提升代码复用性。

用参数初始化表对数据成员初始化

核心作用

简化构造函数，不在函数体内，而在函数首部实现数据成员初始化

语法格式

类名::构造函数名（[参数表]）[:成员初始化表]
{
    [构造函数体]//可选项
}

代码示例（C++）

```cpp
class Box {
private:
    int height;
    int width;
    int length;
public:
    Box(int, int, int);
    int Volume();
};

//Box::Box(int h, int w, int l) {
//    height = h;
//    width = w;
//    length = l;
//}
Box::Box(int h,int w,int l):height(h),width(w),length(l){}
```

关键点总结：
该方法可以减少函数体长度，使结构函数显得精炼简单。在需要初始化的数据成员较多时，能体现优越性。

构造函数的重载与默认参数

构造函数重载（Overloading）

定义：同名函数，参数列表（个数/类型/顺序）不同。
作用：提供多种灵活的对象初始化方式。

```
class Date {
public:
Date() { ... } // 无参构造
Date(int y) { ... } // 单参数
Date(int y, int m, int d) { ... } // 多参数
};
```

构造函数默认参数（Default Params）

定义：声明时为参数指定默认值。
作用：调用时可省略已设默认值的参数。

```
class Date {
public:
Date(int y=2000, int m=1, int d=1) { ... }
};
// 调用: Date d1; Date d2(2024); Date d3(2024,5);
```

核心注意事项：避免歧义

应避免同时使用“重载”和“默认参数”。这可能导致函数调用时产生歧义（Ambiguity），使得编译器无法确定应该调用哪个构造函数，从而引发编译错误。

案例3：VR角色

场景描述：
在VR多人互动中，创建虚拟化身（Avatar）。大部分用户的身高都是默认的（1.7米），少数用户可以自定义身高和名字。
使用默认参数可以减少代码量，避免定义多个重载版本。

```cpp
#include <iostream>
#include <string>
using namespace std;

class VRAvatar {
private:
    string name;
    float height;   // 身高（米）
    float speed;    // 移动速度
public:
    // 默认参数构造函数：如果调用时没传参数，则使用默认值
    VRAvatar(string n = "Guest", float h = 1.70f, float s = 5.0f) {
        name = n;
        height = h;
        speed = s;
        cout << "虚拟化身 " << name << " 已加入世界……" << endl;
    }

    void introduce() {
        cout << "----------------------------------------" << endl;
        cout << "我是 " << name << "，身高 " << height << "m，移动速度 " << speed << " m/s" << endl;
    }
};

int main() {
    VRAvatar player1;                    // 使用全部默认参数
    VRAvatar player2("小明");            // 自定义名字，身高和速度使用默认
    VRAvatar player3("篮球运动员", 2.05f); // 自定义名字和身高，速度使用默认
    VRAvatar player4("闪电侠", 1.80f, 12.0f); // 全部自定义

    player1.introduce();
    player2.introduce();
    player3.introduce();
    player4.introduce();

    return 0;
}
```

析构函数详解

DESTRUCTOR DETAILED EXPLANATION

析构函数的定义与特性

定义：与构造函数作用相反的特殊成员函数，在对象生命周期结束时被自动调用，用于执行对象的清理工作。

核心特性

命名规则

类名前加波浪号 `~`，如 ~ClassName()

无参无返

不能有参数和返回值，因此无法重载

自动调用

对象销毁时自动执行（作用域结束/delete）

主要作用（资源释放）

释放内存

释放动态分配的堆内存，防止泄漏

关闭文件

关闭对象打开的文件句柄，释放资源

断开连接

释放网络、数据库等系统级连接资源

案例4：析构函数的应用

本案例展示了如何在析构函数中释放通过**new**动态分配的内存，避免内存泄漏，这是析构函数最核心的用途之一。

```cpp
#include<iostream>
#include<string>
using namespace std;
class Student {
public:// 构造函数：分配资源
    Student(int n, string nam) : num(n), name(nam) {
        cout << "构造函数：对象创建成功。" << endl;
    }
    ~Student() {// 析构函数：释放资源（重点）
        cout << "析构函数：释放内存资源。" << endl;
    }
private:int num; string name;
};

int main() {
    Student* p = new Student(10010, "Li Lin");// 动态分配内存
    delete p;// 自动调用析构函数
    return 0;
}
```

关键机制：当使用**delete**运算符删除动态对象时，系统会自动调用该对象的析构函数来完成清理工作。

举一反三案例2：VR资源的释放

应用场景痛点

VR应用中纹理常驻GPU显存，若不及时释放会导致显存泄漏，严重影响帧率甚至引发程序崩溃。

RAII 资源管理方案

封装VRTexture类：构造函数加载并上传纹理至GPU；析构函数自动调用API释放显存资源，确保生命周期安全。

VRTexture.h / C++ Implementation

```
class VRTexture {
public:
VRTexture(const string& path) {
glGenTextures(1, &m_id);// 创建GPU纹理
// ... 上传数据 ...
}
~VRTexture() {
// 自动释放GPU资源
glDeleteTextures(1, &m_id);
}
private: GLuint m_id;
};
```

核心价值：利用 C++ 析构函数的自动调用机制，将 GPU 资源的释放与对象生命周期绑定，彻底避免手动管理带来的内存泄漏风险。

构造与析构的调用顺序

Calling Sequence of Construction and Destruction

构造与析构的调用顺序规则

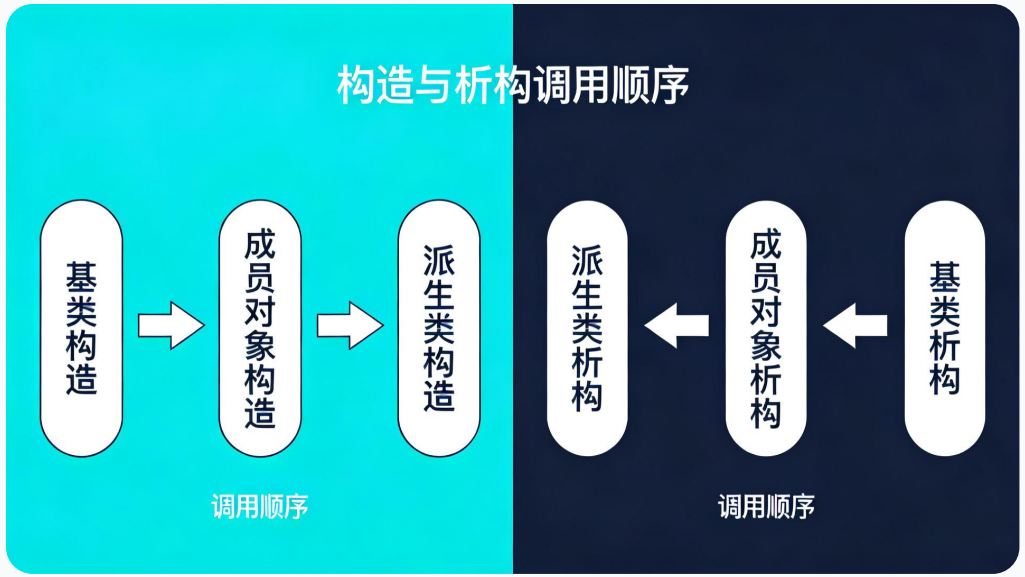
构造顺序（Constructor Order）

1. **基类构造**：先调用基类的构造函数。
2. **成员对象**：按声明顺序调用成员对象构造。
3. **派生类**：最后调用派生类自身构造体。

析构顺序（Destructor Order）

析构函数的调用顺序与构造函数的调用顺序完全相反。

核心口诀：先构造的后析构，后构造的先析构



图示：构造与析构调用顺序流程对比

Note：构造函数用于初始化对象状态，析构函数用于清理资源，两者调用顺序严格遵循栈式原则。

先构造的后析构，后构造的先析构

```cpp
#include <iostream>
#include <string>
using namespace std;

class VRAvatar {
private:
    string name;
    float height;   // 身高（米）
    float speed;    // 移动速度
public:
    // 默认参数构造函数：如果调用时没传参数，则使用默认值
    VRAvatar(string n = "Guest", float h = 1.70f, float s = 5.0f) {
        name = n;
        height = h;
        speed = s;
        cout << "虚拟化身 " << name << " 已加入世界……" << endl;
    }

    ~VRAvatar() {
        cout << "析构函数被调用" <<name<< endl;
    }
    void introduce() {
        cout << "-------------------------------------------" << endl;
        cout << "我是 " << name << "，身高 " << height << "m，移动速度 " << speed << " m/s" << endl;
    }
};

int main() {
    VRAvatar player1;                   // 使用全部默认参数
    VRAvatar player2("小明");           // 自定义名字，身高和速度使用默认
    VRAvatar player3("篮球运动员", 2.05f); // 自定义名字和身高，速度使用默认
    VRAvatar player4("闪电侠", 1.80f, 12.0f); // 全部自定义

    player1.introduce();
    player2.introduce();
    player3.introduce();
    player4.introduce();

    return 0;
}
```

案例5：调用顺序演示

通过综合案例代码与运行结果对照，直观解析构造与析构函数的调用顺序逻辑。

C++ 代码实现

```
classBase{public:Base() {cout<<"Base 构造"; } };
classMember{public:Member() {cout<<"Member 构造"; } };
classDerived:publicBase{Memberm;public:Derived()
{cout<<"Derived 构造"; } };
intmain() {Derivedd;return0; }
```

运行结果与解析

1. Base 构造函数调用// 先构造基类
2. Member 构造函数调用// 再构造成员对象
3. Derived 构造函数调用// 最后构造派生类
4. Derived 析构函数调用// 先析构派生类
5. Member 析构函数调用// 再析构成员对象
6. Base 析构函数调用// 最后析构基类

核心规律：构造时由“基”到“成”再到“派”；析构时严格遵循相反顺序，确保资源正确释放。

对象数组

OBJECT ARRAYS

掌握对象数组的定义与使用，高效管理复杂数据结构

从理论基础到实战应用，构建稳固的编程逻辑体系

什么是对象数组？

核心定义

数组的每个元素不再是简单数据类型，而是某个类的实例对象。

主要用途

高效管理一组同类型对象，如：班级学生列表、游戏敌人集合。

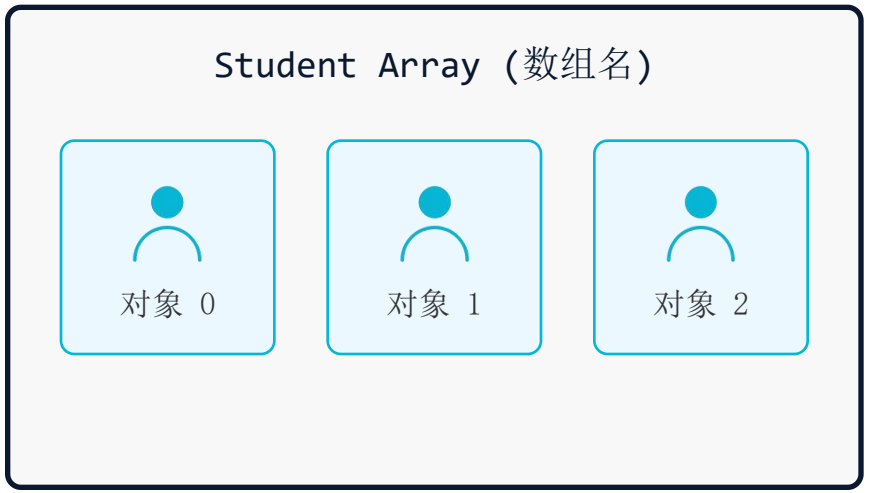
声明语法

类名 数组名[数组大小];(如: Student stu[5];)

初始化规则

- 有默认构造函数：自动调用初始化
- 无默认构造函数：必须显式调用带参构造

对象数组模型示意



每个索引位置存储的是一个完整的对象实例，包含该对象的所有属性和方法。

案例：对象数组的初始化与使用（例9.6）

```
class Box {
public:
Box(int h=10, int w=10, int len=10) : height(h),
width(w), length(len) {}
int volume() { return height * width * length; }
private:
int height, width, length;
};
int main() {
// 创建对象数组并初始化
Box boxes[3] = {Box(10,12,15), Box(15,20,25),
Box(20,25,30)};
for(int i=0; i<3; i++)
cout << "Volume: " << boxes[i].volume() << endl;
return 0;
}
```

对象数组初始化

通过构造函数列表直接初始化数组元素，每个元素都是一个独立的 Box 对象，包含完整的属性和方法。

成员访问与计算

使用下标运算符 `[]` 访问数组中的对象，通过点运算符 `.` 调用成员函数 `volume()` 计算体积。

核心要点：

对象数组是 C++ 中管理多个同类型对象的高效方式。本例展示了如何在栈上创建对象数组，利用构造函数初始化状态，并通过循环遍历统一处理数据。

举一反三案例：VR场景中的道具数组

应用场景

在VR游戏开发中，玩家收集的武器、药水、钥匙等多种类型道具，通过对象数组进行统一管理和遍历。

实现逻辑

定义VRItem类封装道具属性，利用数组存储多个实例，通过循环遍历展示玩家背包状态，便于扩展和维护。

```
classVRItem {// 定义道具类
public:string name, type, effect;
void ShowInfo() {/* 展示信息 */}
};

int main() {
// 创建包含3个道具的数组
VRItem items[3] = {
VRItem("能量剑","武器","伤害100"),
VRItem("生命药水","消耗品","回血50"),
VRItem("传送钥匙","钥匙","开门")
};
// 遍历数组显示所有道具
for(inti=0; i<3; i++) {
 items[i].ShowInfo();
}
  return 0;
}
```

核心价值：对象数组将“数据结构”与“业务逻辑”完美结合。通过数组实现批量管理，通过类封装实现数据安全与行为内聚，这是面向对象编程在游戏开发中的典型实践。

虚拟现实中的应用案例

CHAPTER 5: APPLICATION CASES IN VIRTUAL REALITY

课程总结与回顾

构造函数（Constructor）

对象初始化的关键，支持无参、带参、重载和默认参数，确保对象在创建时处于有效状态。

析构函数（Destructor）

对象清理的保障，在对象销毁时自动调用，用于释放资源（如内存、文件句柄），防止内存泄漏。

调用顺序规则

构造顺序：基类 → 成员对象 → 派生类；析构顺序与构造相反，严格遵循“先构造后析构”原则。

感谢观看！

Q & A

欢迎提问与交流